Lightweight Heuristic Data Selection and Static Curriculum Ordering for Compute-Efficient LoRA Fine-Tuning of a Sub-1B Language Model

Author: Bezawit Assefa

Status: working draft (single-seed controlled study; descriptive findings only)

1. Abstract

Fine-tuning generative language models on large instruction corpora is expensive, and data-centric methods promise comparable quality from smaller subsets. We present a controlled empirical investigation of whether lightweight heuristic data selection and static curriculum ordering can retain the benefit of full-data instruction tuning at reduced training cost in a sub-1B setting. Using Qwen2.5-0.5B-Instruct with LoRA (r=8, α=16) on the Alpaca dataset (49,401 training examples), we cross two factors — subset selection (full, uniform random 50%, adaptive top-50% by a heuristic importance score combining embedding-based diversity, instruction complexity, and response length) and ordering (random vs. static easy-to-hard curriculum) — for seven single-seed runs on one NVIDIA T4. Evaluation is held-out full-sequence language-model loss. Random 50% selection increased held-out LM loss by only +0.0113 (+0.96%) relative to full-data training while reducing measured training wall-clock time by approximately 49%. In contrast, adaptive heuristic selection performed worse than random selection at equal budget (+0.0088), and the static curriculum produced no benefit distinguishable from measurement noise in either selection condition; component ablations differed from the combined score by less than the observed same-seed repeat variation (~2×10^{-5}). Adaptive subsets showed lower training loss but higher evaluation loss, consistent with selection of easier-to-fit or distribution-shifted examples — a hypothesis-generating observation, not a proven explanation. Findings are descriptive (single seed) and specific to this setting.

2. Introduction

Instruction tuning improves the usability of pretrained language models, but its cost scales linearly with the number of training examples per epoch. A natural efficiency question is whether all training examples contribute equally to learning, or whether a well-chosen half of the corpus — possibly presented in a deliberate order — can retain most of the benefit at roughly half the cost.

This question has been studied mostly at larger scales. Data-selection work such as DEITA demonstrates large reductions at 7B–13B scale using LLM-based scorers [1], while curriculum-learning studies explore difficulty-based ordering, often with dynamic or utility-weighted signals [2, 3]. Whether *lightweight* heuristic scoring combined with *static* ordering helps at sub-1B parameter-efficient fine-tuning (PEFT) scale has received less attention.

We address this gap with a controlled, scoped experiment. We fine-tune Qwen2.5-0.5B-Instruct with LoRA on Alpaca under seven conditions that cross subset selection (full data, uniform random 50%, adaptive top-50% by an interpretable heuristic importance score) with presentation order (uniform shuffle vs. static easy-to-hard curriculum). Two component ablations isolate the

diversity and complexity signals of the scorer. All runs share the same seed, split, hyperparameters, hardware session, and held-out evaluation set, so observed differences are attributable to the manipulated factors within this setup.

Our strongest defensible finding is an efficiency result: a uniformly random 50% subset increased held-out full-sequence language-model loss by +0.0113 (+0.96%) relative to the full corpus while reducing measured training time by approximately 49%. The proposed adaptive selection and static curriculum provided no benefit distinguishable from measurement noise over random selection in this setting. Each cell was trained once; conclusions are therefore descriptive.

The contribution is deliberately modest: a controlled selection × ordering factorial pilot at sub-1B LoRA scale using lightweight, training-free heuristics. It provides (a) a measured efficiency anchor point — random 50% of Alpaca retained held-out LM loss within +0.96% of full-data training while roughly halving measured wall-clock cost in this setup — and (b) a documented cautionary negative result on heuristic adaptive selection and static easy-to-hard ordering. The complete artifact trail (executed notebook, canonical JSON results, figure-generation script), including disclosed provenance limitations, accompanies the study so that every reported number can be independently checked.

3. Research Questions and Hypotheses

Central research question. Can lightweight heuristic data selection and static curriculum ordering retain the benefit of full-data instruction tuning while reducing training cost in a sub-1B parameter LoRA fine-tuning setting?

This decomposes into four questions addressed by planned contrasts:

- **RQ1 (data volume).** How much held-out LM loss is incurred by halving the training corpus? Contrast: E1 vs. E2.
- **RQ2 (selection).** At equal budget (50%), does adaptive heuristic selection outperform uniform random selection? Contrast: E2 vs. E3.
- **RQ3 (ordering).** Does static easy-to-hard curriculum ordering add measurable benefit on top of selection, or without it? Contrasts: E3 vs. E4 (adaptive subsets), E2 vs. E5 (random subsets).
- **RQ4 (score components).** Do individual scorer components account for the combined score's behavior? Contrast: E3 vs. A1/A2 ablations.

Pre-stated hypotheses (stated prior to examining the outcome contrasts):

- H1: a 50% subset will incur a small increase in held-out LM loss relative to the full corpus.
- H2: adaptive selection will match or outperform random selection at equal budget (motivated by DEITA-style findings at larger scale [1]).
- H3: curriculum ordering will add a small benefit on top of either selection strategy.
- H4: no single scorer component alone will explain the combined score's behavior.

Outcomes in this setting: H1 was observed to hold; H2 and H3 were not supported by the data (no benefit distinguishable from measurement noise was observed); H4 was not meaningfully testable because the ablations differed from the combined score by less than the observed same-seed repeat variation ($\sim 2 \times 10^{-5}$).

4. Related Work

4.1 Data Selection for Instruction Tuning

Data-centric selection aims to identify the smallest effective training subset. DEITA (Liu et al., 2024) scores instructions on complexity, quality, and diversity using trained scorer models and selects a minimal subset: performance comparable to training on all 300K pool samples was achieved with only 3K selected samples (a reported ~100× reduction), and models trained on 6K automatically selected samples performed on par with or better than open-source alignment models using over 10× more data [1]. DEITA selects a subset but does not test whether the presentation *order* of that subset adds further benefit, and its LLM-based scorers are not necessarily feasible at sub-1B PEFT scale. Surveys of data selection for LLM instruction tuning frame the space of quality scoring, diversity/coverage, and influence-based methods [5]; we use this literature for framing rather than as experimental baselines.

Our scoring function is DEITA-inspired but deliberately lightweight: embedding distance, surface complexity proxies, and a response-length prior replace trained scorer models, making the pipeline feasible alongside PEFT on a single commodity GPU.

4.2 Curriculum Learning and Data Ordering for LLMs

Curriculum learning presents examples ordered by estimated difficulty. Recent work complicates the simple easy-to-hard recipe: DUCL (Jiang et al.) argues that pure difficulty-based curricula are insufficient and combines difficulty with a per-sample utility signal [2]; EDCO (Pang et al.) adapts the order dynamically during training rather than fixing it beforehand [3]. Domain-specific studies (e.g., medical QA) find that curriculum effects depend strongly on the difficulty signal and domain, and are often modest [4].

Our method is intentionally the *static*, pure-complexity baseline case of this design space. The results below are consistent with the sensitivity reported in this literature: with a shallow difficulty signal and a fixed order, we observed no curriculum effect that could be distinguished from measurement noise in our single-seed setting. This does not contradict curriculum approaches that use richer or adaptive signals.

4.3 Parameter-Efficient Fine-Tuning

LoRA freezes pretrained weights and injects trainable low-rank matrices into attention (and, here, MLP) projections, reducing trainable parameters by orders of magnitude while remaining competitive with full fine-tuning in many settings [6]. We extend LoRA into a selection-plus-ordering context it was not originally designed for, at 0.5B scale.

4.4 Gap Statement

DEITA-style work establishes that complexity-, quality-, and diversity-based selection can improve instruction-tuning efficiency at 7B–13B scale, achieving near-full performance with a small fraction of the data; however, this line of work has generally not examined whether the *order* in which selected examples are presented adds further benefit beyond selection alone. Separately, the curriculum literature explores difficulty-based ordering but typically without combining it with principled data selection, and largely not at sub-1B PEFT scale. The combination — lightweight

heuristic selection evaluated jointly with static ordering, isolated in a controlled factorial design at sub-1B scale — has received less attention. This paper addresses that combination directly.

5. Methodology

This section summarizes the implementation-faithful methodology; `paper/methodology.md` is the authoritative, fully detailed specification written from the source code (`src/`, `configs/`).

5.1 Problem Formulation

Supervised instruction tuning fine-tunes a pretrained model on (instruction, input, response) triples. We study two orthogonal interventions on a corpus of N = 49,401 training examples:

- **Selection** — choosing a subset S ⊆ D with |S| = f·N (f = 0.5 for all subset experiments); strategies: full corpus (f = 1), uniform random subsampling, and importance-score-based top-k ("adaptive") selection.
- **Ordering** — presenting S as a uniformly shuffled sequence ("random") or sorted by increasing estimated difficulty ("curriculum").

The design crosses these factors so each factor's effect is isolable at a fixed compute budget.

5.2 Importance Scoring

Every training example receives three sub-scores, min–max normalized to [0, 1] over the training split, combined as:

$$\text{Importance}(x) = \alpha \cdot D(x) + \beta \cdot C(x) + \gamma \cdot R(x), \text{ with } \alpha = 0.333,\ \beta = 0.333,\ \gamma = 0.334,$$

weights fixed a priori (uniform thirds) and never tuned against any evaluation result. No LLM judge is used anywhere in the pipeline.

Diversity D(x). Each example's instruction (with input, if present) is embedded with `all-MiniLM-L6-v2` (unit-normalized). $D(x) = 1 - e_i^{\top}\mu$ against the corpus centroid μ, then min–max normalized; high scores indicate examples far from the distributional center.

Complexity C(x). From the instruction concatenated with its input field: character length ℓ(x) and vocabulary richness v(x) (unique whitespace-delimited lowercase tokens / total tokens). Each signal is min–max normalized independently, averaged, and min–max normalized again. This is deliberately a shallow proxy responding to surface length and lexical variety.

Response-length R(x). With r(x) the gold-response character length, the implemented score is $R(x) = \text{clip}_{[0,1]}\left(1 - \frac{|\text{minmax}(\log(1+r))(x) - t|}{\max(t,\, 1-t)}\right)$ with target $t = 0.6$ (so the denominator equals 0.6): log(1+r) is min–max normalized over the training split, the score peaks (=1) where that normalized length equals t, and it decays linearly to 0 at both normalized endpoints — i.e., at the shortest and longest responses in the split. This favors mid-range response lengths (~200–800 characters in Alpaca). The target was chosen a priori and not tuned.

Scores are computed once over the training split and cached; the held-out set never participates in scoring or selection.

5.3 Selection

Three mechanisms are implemented: (1) full-data selection (f = 1.0); (2) uniform random selection of k = ⌊f·N⌋ = 24,700 examples via `random.Random(seed)` without replacement; (3) adaptive selection retaining the top k examples by descending importance score, with deterministic tie-breaking by stable sort.

5.4 Ordering / Curriculum

Given the selected subset: random ordering is a seeded uniform shuffle; the easy-to-hard curriculum sorts ascending by the complexity sub-score C(x). Both orderings are **static**: fixed before training, identical across epochs. There is no dynamic difficulty estimation, re-scoring, or pacing schedule. The curriculum signal is the same heuristic complexity measure used inside the selection score; this non-orthogonality between selection and ordering signals is acknowledged as a design property.

5.5 Loss Formulation (Important for Interpretation)

Training and evaluation use **full-sequence language-modeling loss**: token labels cover the complete rendered sequence (instruction, optional input block, response, EOS). Instruction tokens are **not** masked; the model receives cross-entropy supervision on every non-padding token, including prompts. Consequently, `eval_loss` measures how well the adapted model predicts held-out Alpaca sequences — including their instructions — under the same objective used in training. It is **not** a direct measure of instruction-following quality, fluency, factual accuracy, or generation adequacy; no generation was sampled and no generation-based metric (ROUGE/BLEU/win-rate/human judgment) was computed anywhere in this study. Because the objective and evaluation set are identical across all conditions, relative comparisons remain internally meaningful; absolute values should not be compared to studies using completion-only masking.

6. Experimental Setup

6.1 Dataset

`tatsu-lab/alpaca` [7] (52,002 examples) is loaded via Hugging Face `datasets`; a single random split (test_size = 0.05, seed = 42) yields 49,401 training and 2,601 held-out evaluation examples, shared identically by all seven runs. No deduplication, filtering, or cleaning beyond template rendering is applied. Examples are rendered into a fixed prompt template containing the instruction, optional input block, and target response.

6.2 Model and Training Configuration

Component	Value
Base model	Qwen/Qwen2.5-0.5B-Instruct (498,431,872 parameters)
Adaptation	LoRA (PEFT), causal-LM
LoRA rank / alpha / dropout	8 / 16 / 0.05
Target modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj
Trainable parameters	4,399,104 (0.8826%)
Optimizer	AdamW (`adamw_torch`)
Learning rate / schedule	2×10^{-4} / cosine decay, 3% warmup
Weight decay	0.001
Epochs	1
Micro-batch × grad. accumulation	8 × 4 (effective batch 32)
Max sequence length	384 tokens (truncation; dynamic padding, labels masked at −100)
Precision	FP16 mixed precision
Gradient checkpointing	Enabled
Seed	42

The base model is frozen; only LoRA adapters receive gradients. Training uses Hugging Face `Trainer`.

6.3 Experimental Design

Seven runs share model, LoRA configuration, hyperparameters, data split, evaluation set, and seed; only selection strategy, fraction, and ordering differ.

Main experiments (selection × ordering):

Experiment	Selection	Fraction	Ordering	Role
E1	Full data	1.00	Random	Upper reference; efficiency anchor
E2	Uniform random	0.50	Random	Control: effect of halving data volume
E3	Adaptive top-50%	0.50	Random	Effect of scored selection at equal budget
E4	Adaptive top-50%	0.50	Easy→hard	Proposed full method (selection + ordering)
E5	Uniform random	0.50	Easy→hard	Isolates ordering without selection

Planned contrasts: E2 vs. E3 (selection), E3 vs. E4 (ordering on adaptive subsets), E2 vs. E5 (ordering on random subsets), E1 vs. E2 (cost of halving the corpus).

Component ablations: A1 (α=1, β=γ=0: diversity-only) and A2 (β=1, α=γ=0: complexity-only), both adaptive 50%, random order. These are ablations of E3′s selection rule, **not** additional cells of the main selection × ordering matrix. Both share E3′s fraction and ordering for direct comparison. The response-length component (γ alone) was not ablated separately.

6.4 Efficiency Measurement

All canonical runs executed sequentially on a single NVIDIA Tesla T4 (Kaggle session), FP16. Per run we record:

- **Wall-clock training time**: `time.perf_counter()` bracketing `Trainer.train()` only — including forward/backward passes, gradient accumulation, optimizer steps, gradient-checkpoint recomputation, and periodic logging; excluding downloads, loading, tokenization, scoring, subset construction, checkpoint serialization, and the post-training evaluation pass.
- **Peak GPU memory**: `torch.cuda.max_memory_allocated()`, reset before training and read after evaluation completes (so the reported peak spans training plus the final evaluation pass).
- **System RAM** via psutil as auxiliary telemetry.

Timing comparisons are between runs on identical hardware and session type; absolute times do not extrapolate to other devices.

7. Results

All numbers below are taken exactly from the canonical artifacts (`outputs/**/results.json`, aggregated in `outputs/experiment_results.csv` and `outputs/all_results.json`). Eval loss is **held-out full-sequence language-model loss**, not a generation-quality metric (§5.5).

7.1 Main Experiments

Table 1 — Main experiments (seed 42; losses at 6 decimal places; full precision in artifacts).

Exp	Selection	Ordering	Fraction	Train examples	Eval loss ↓	Train loss	Time (min)	Time red. vs E1
E1	Full	Random	1.00	49,401	1.184476	1.221839	65.22	—
E2	Random	Random	0.50	24,700	1.195796	1.236928	33.26	49.0%
E3	Adaptive	Random	0.50	24,700	1.204629	1.154256	28.38	56.5%
E4	Adaptive	Curriculum	0.50	24,700	1.204174	1.154270	28.44	56.4%
E5	Random	Curriculum	0.50	24,700	1.195706	1.236344	33.24	49.0%

Main contrasts (eval-loss differences; percentages relative to the earlier run):

Contrast	Factor isolated	Δ eval loss	Relative
E1 ⟶ E2	halving data (random)	+0.0113199949	+0.9557%
E2 ⟶ E3	adaptive vs. random selection	+0.0088328123	+0.7387%
E3 ⟶ E4	curriculum on adaptive subsets	−0.0004551411	−0.0378%
E2 ⟶ E5	curriculum on random subsets	−0.0000904799	−0.0076%

Figure 1 visualizes held-out evaluation loss across all seven conditions, and Figure 2 plots the efficiency frontier — evaluation loss against measured wall-clock training time.

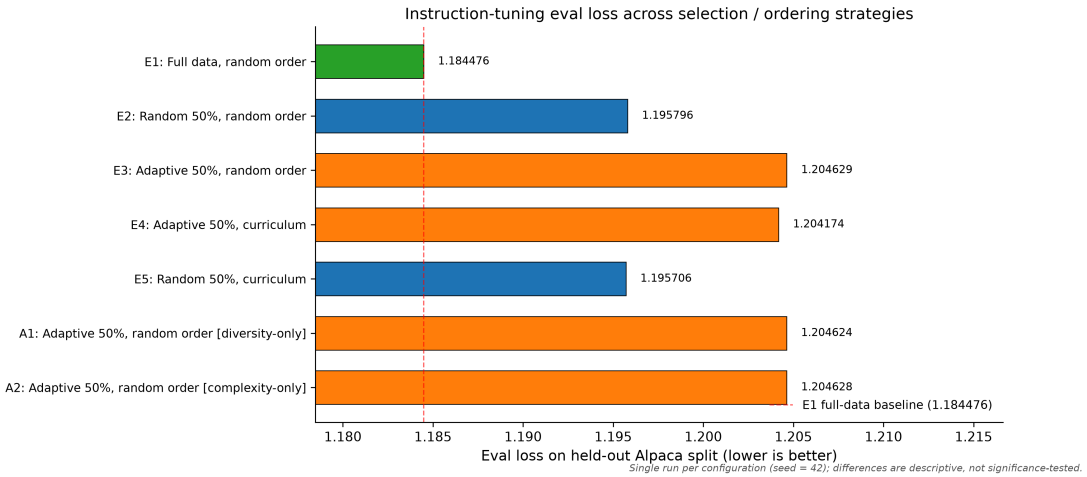

Figure 1: Held-out evaluation loss across all seven experimental conditions (lower is better). Colors distinguish the full-data baseline (green), uniform-random 50% subsets (blue squares), and adaptively selected 50% subsets (orange triangles); diamond markers denote the two scoring-component ablations (A1, A2). All bars are single-seed runs on identical splits and hyperparameters; differences smaller than the observed same-seed repeat variation (~2×10^{-5} eval loss) should not be ranked.

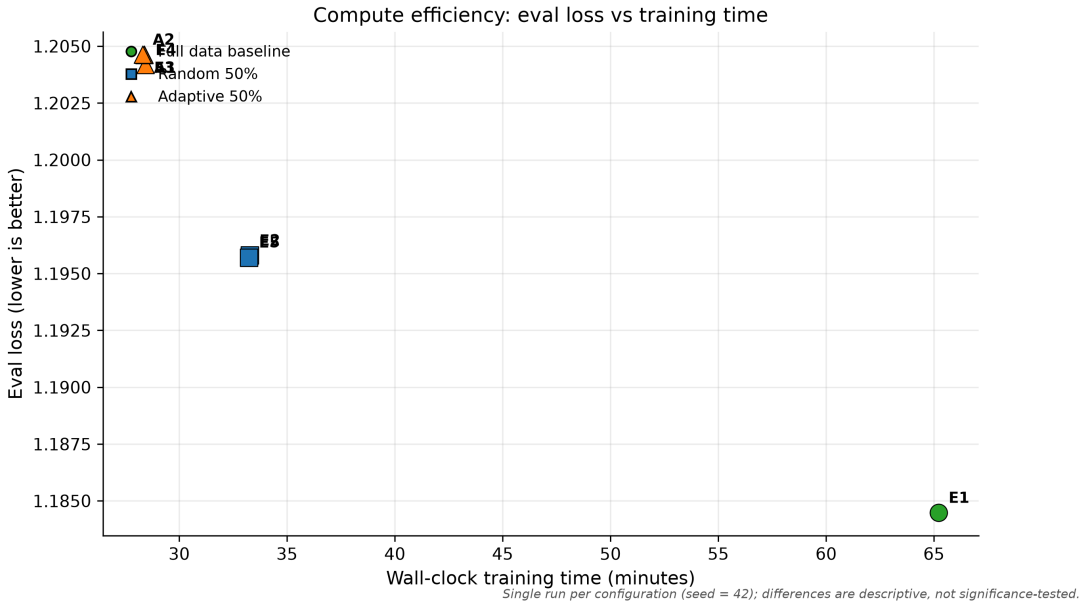

Figure 2: Efficiency frontier: held-out evaluation loss versus measured wall-clock training time (minutes, single NVIDIA T4). Random 50% subsets (blue squares) approach full-data quality at roughly half the training cost, while adaptive 50% subsets (orange triangles) train faster still but evaluate worse than random in this setup; the full-data baseline is shown in green.

Observations (descriptive; n = 1 per cell):

- Halving the corpus uniformly at random increased held-out LM loss by +0.0113 (+0.96%) while reducing measured training time by ~49.0% (65.22 → 33.26 min). Random 50% retained performance close to the full-data baseline in this setup.
- Adaptive 50% selection performed worse than uniform random 50% selection in this experimental setting (+0.0088 eval loss at equal budget). Among the strategies tested in this pilot, uniform random selection performed best.
- No curriculum benefit distinguishable from practical measurement resolution was observed for the tested static curriculum in either selection condition: E3→E4 changed eval loss by −0.0005 (−0.038%) and E2→E5 by −0.0001 (−0.008%). Both deltas are far smaller than the selection effect; E2→E5 sits within a few multiples of the same-seed repeat difference discussed in §10, and even the larger E3→E4 delta is below any threshold of practical significance that a single-seed design can support.

A notable pattern: adaptive runs show **lower training loss** (≈1.1543) than random/full runs (≈1.2218–1.2369) despite **higher** evaluation loss. We return to this in §8.

7.2 Component Ablations

Table 2 — Scoring-component ablations (adaptive 50%, random order; ablations of E3′s selection rule, not cells of the main matrix).

Exp	Score weights	Eval loss ↓	Train loss	Δ vs. E3
E3 (reference)	α=β=0.333, γ=0.334	1.204629	1.154256	—
A1	α=1, β=γ=0 (diversity-only)	1.204624	1.154256	−0.0000057220
A2	β=1, α=γ=0 (complexity-only)	1.204628	1.154260	−0.0000016689

Both ablations selected subsets whose training outcomes were numerically almost identical to E3′s: differences of ~6×10^{-6} and ~2×10^{-6} eval loss respectively. These differences fall **within the observed same-configuration repeat variation** (§10: two same-seed E1 executions differed by ~2×10^{-5} eval loss) and therefore must not be ranked or interpreted as evidence about component contributions. What can be said descriptively is that replacing the combined score with either component alone produced essentially the same outcome in this setup; H4 could not be meaningfully tested because there was no combined-score effect to decompose.

7.3 Provenance Note on Repeated Baseline Runs

Two genuine E1 executions exist (identical configuration and seed): a standalone validation run (eval loss 1.184454, 68.62 min) and the suite run reported here (1.184476, 65.22 min). The suite run is canonical because it belongs to the complete executed seven-run suite. The pair differs

by ~2×10^{-5} eval loss; this is an **observed same-seed repeat difference**, not a formal variance estimate, and it defines the practical resolution below which numerical differences in this study should not be interpreted.

8. Discussion

The headline efficiency finding. In this setup, discarding half of Alpaca uniformly at random cost +0.96% held-out LM loss and saved ~49% of measured training time. For practitioners operating in regimes similar to this one (sub-1B model, LoRA, one epoch, ~50k generic instruction examples), this data point is consistent with dataset size not being the binding constraint on held-out loss in this setting — echoing, with a purely random control at sub-1B scale, data-quality findings reported at larger scales [1]. The efficiency relationship underlying this observation is visualized in Figure 2.

The negative results, stated cautiously. The hypothesized benefits of adaptive selection (H2) and static curriculum ordering (H3) were not observed: adaptive selection performed worse than random selection at equal budget, and curriculum changed eval loss by less than ±0.04% in both selection conditions. We emphasize what these statements do and do not claim. They describe one scorer, one ordering rule, one model, one dataset, and one epoch. They do not support claims that adaptive selection generally hurts performance, that random selection is universally superior, that curriculum learning does not work for LLMs, or that heuristic scoring is ineffective in general — particularly given published evidence that curriculum gains depend heavily on the difficulty signal and domain [2–4] and that stronger selection signals succeed at other scales [1].

Why might adaptive selection have underperformed? A hypothesis-generating observation. The most instructive pattern in Table 1 is the divergence between training and evaluation loss: adaptive runs fit their training data better (train loss ≈ 1.1543 vs. ≈ 1.2218–1.2369) yet evaluate worse than random runs. One consistent-with-the-data interpretation is that the scorer preferentially selected examples that were *easier for the adapted model to fit* and/or shifted the training distribution away from the held-out distribution — i.e., the score correlates with fitting ease rather than training utility. We offer plausible contributing mechanisms, none of which is experimentally proven here:

- **Scorer/model mismatch.** All three heuristics are computed without reference to the model being trained; features that look important may simply be easy to fit (e.g., long, lexically rich instructions produce more supervised tokens of predictable text).
- **Heuristic proxies vs. training utility.** Surface complexity and centroid-distance diversity need not correspond to examples that improve generalization; DEITA-style pipelines additionally use learned quality scorers, which our lightweight design omits [1].
- **Corpus redundancy/noise.** Alpaca contains duplicated and templated examples; top-50% selection may concentrate redundant or degenerate patterns that lower training loss without helping held-out prediction.
- **Diversity definition.** Distance from the embedding centroid rewards outlier directions; outlier-heavy subsets can be distribution-shifted relative to the dense core that the held-out sample represents.

- **Response-length bias.** The bell-shaped reward encodes an a-priori assumption about informative response lengths; if incorrect for this corpus, it injects systematic selection bias.
- **Shared signal between selection and ordering.** The curriculum sorts by the same complexity feature used in selection, so the two interventions are not independent signals; any ordering effect is entangled with how complexity shaped the subset.

This pattern — lower train loss with higher eval loss under score-based selection — is the study's most transferable cautionary observation and a concrete target for follow-up work (e.g., model-informed or influence-based scores, utility-weighted curricula [2], deduplicated corpora, multiple seeds).

What would strengthen or revise these conclusions? Multi-seed replication to establish real variance; completion-masked and generation-based evaluations to check whether held-out LM loss tracks instruction-following quality; stronger scorers (including LLM-judge or influence-function scores); dynamic curricula [3]; and other models, datasets, and epochs. Until such evidence exists, the appropriate summary is: in this controlled sub-1B setting, the simplest strategy fared best among those tested — random halves matched or beat scored halves on this metric in every tested contrast in this single-seed setup, and ordering added nothing observable.

9. Limitations

1. **Single seed.** n = 1 per experimental cell; no significance testing is possible or claimed.
2. **One model.** Qwen2.5-0.5B-Instruct only; findings may not transfer to other architectures or scales.
3. **One dataset.** Alpaca only; a duplicated, English, single-task-style instruction corpus.
4. **One epoch.** Selection/ordering effects can interact with training duration; longer schedules were not tested.
5. **One LoRA configuration.** r = 8, α = 16, dropout 0.05, seven projection targets; no sweep.
6. **No tuning of scoring weights.** Uniform thirds were fixed a priori; the sensitivity of adaptive selection to (α, β, γ) is unknown.
7. **Incomplete ablation coverage.** The response-length component (γ) was not ablated independently.
8. **Non-orthogonal curriculum signal.** Curriculum ordering uses the same complexity feature as selection, confounding attribution.
9. **Metric scope.** Evaluation is held-out **full-sequence LM loss** only; instruction tokens are supervised, so absolute values are not comparable to completion-masked studies.
10. **No generation/human/downstream evaluation.** No ROUGE/BLEU/win-rate, human judgment, downstream benchmarks, factuality, or safety assessment; lower eval loss does not demonstrate better instruction-following.
11. **Dataset revision not pinned.** The HF dataset revision hash was not recorded.
12. **Model revision not pinned.** The HF model/tokenizer revision hash was not recorded.
13. **Environment not locked.** Exact package versions were not captured; requirements state minimum versions only.

14. **Narrow timing scope.** Wall-clock brackets `Trainer.train()` only; end-to-end cost (scoring, tokenization, evaluation) is excluded and scoring itself has nonzero cost not reflected in the reported times.
15. **Variance uncharacterized.** One same-config repeat exists; a single repeat pair cannot estimate variance.
16. **Checkpoints not retained.** Model checkpoints were intentionally not preserved; behavioral analysis beyond logged metrics is impossible post hoc.

10. Reproducibility

What supports reproduction.

- **Seeding:** global seed 42 applied to Python `random`, NumPy, and PyTorch (CPU + CUDA) with deterministic cuDNN flags; ordering uses seed + 1; every experiment uses seed 42.
- **Version-controlled configs:** all hyperparameters live in YAML files (`configs/base_config.yaml`, `configs/exp*.yaml`, `configs/ablation_*.yaml`) merged at launch.
- **Single entry point:** `src/train_baseline.py --config <experiment>.yaml` executes scoring → selection → ordering → training → evaluation → artifact writing end-to-end.
- **Executed record:** `notebooks/notebook623351e51a.ipynb` preserves the complete executed Kaggle session (logs and outputs) for all seven canonical runs. It contains Kaggle-specific paths and is an execution record, not a portable runner; `notebooks/cloud_run.ipynb` is the portable orchestration notebook for re-running the suite.
- **Canonical artifacts:** full-precision per-experiment JSON (`outputs/<experiment>/results.json`) plus aggregations `outputs/experiment_results.csv` and `outputs/all_results.json`, verified for exact agreement.
- **Figures:** `scripts/generate_figures.py` regenerates all five figures (PNG + PDF) solely from the result artifacts; no metric values are hard-coded.
- **Provenance reconciliation:** the canonical records were reconstructed from the executed notebook and cross-validated; two corrections are documented (adoption of the suite-run E1 as canonical over an earlier standalone E1; recovery of A1′s loss values after a reconstruction collision with A2). No training was rerun during correction.

Disclosed limitations. (i) A1′s `system_ram` value could not be independently recovered from the executed notebook and should not be treated as verified provenance (its losses, time, and GPU memory are notebook/log-supported). (ii) HF dataset and model revisions are unpinned. (iii) Exact environment versions are not locked; bit-level reproduction is not guaranteed. (iv) Run-to-run variation was not systematically characterized; the single same-config E1 repeat pair ($\Delta \approx 2\times10^{-5}$ eval loss) documents observed repeatability at this scale but is not a variance estimate. We therefore do not claim perfect reproducibility; we claim a documented, internally consistent, fully traceable result trail.

11. Conclusion

We conducted a controlled, single-seed factorial study of lightweight heuristic data selection and static curriculum ordering for compute-efficient LoRA fine-tuning of a 0.5B-parameter instruction-tuned model. In this setting, uniformly random 50% subsets retained held-out full-sequence

LM loss within +0.96% of full-data training while reducing measured training time by ~49%; heuristic adaptive selection performed worse than random selection at equal budget; the tested static easy-to-hard curriculum showed no benefit distinguishable from measurement noise; and scorer-component ablations differed from the combined score by less than the observed same-seed repeat variation.

These are descriptive findings from one seed, one model, one dataset, and one epoch. They do not establish that random selection is universally preferable or that heuristic selection fails generally. What they do provide is a cautionary, well-documented sub-1B data point: lightweight DEITA-style heuristics did not translate into measurable gains here, and the accompanying train/eval loss divergence suggests — as a hypothesis for future work — that model-blind importance scores can reward fitability over utility. We release the complete artifact trail (configs, executed notebook, canonical results, figure scripts) to make every number in this paper independently checkable.

12. Appendix

A. Artifact Map

Artifact	Path
Implementation-faithful methodology	`paper/methodology.md`
Configs (base + 7 experiments)	`configs/*.yaml`
Canonical per-run results (full precision)	`outputs/{full_baseline, random50_random_order, adaptive50_random_order, adaptive50_curriculum, random50_curriculum, ablation_diversity_only, ablation_complexity_only}/results.json`
Aggregations	`outputs/experiment_results.csv`, `outputs/all_results.json`
Executed Kaggle notebook (all 7 runs)	`notebooks/notebook623351e51a.ipynb`
Portable orchestration notebook	`notebooks/cloud_run.ipynb`
Figure generation script	`scripts/generate_figures.py`
Figures	`figures/fig1..fig5 (.png/.pdf)`

B. Full-Precision Canonical Values

Exp	eval_loss	train_loss	minutes
E1	1.1844764947891235	1.2218394773611752	65.22
E2	1.1957964897155762	1.236928045440832	33.26
E3	1.2046293020248413	1.1542563734894589	28.38
E4	1.2041741609573364	1.154269584102334	28.44
E5	1.1957060098648071	1.2363436802681247	33.24
A1	1.2046235799789429	1.1542556817049807	28.37
A2	1.2046276330947876	1.1542601288909122	28.31

Peak GPU memory: 6887.5–6887.6 MB across all runs. Standalone E1 repeat (non-canonical, documented): eval loss 1.1844538450241089, 68.62 min.

C. Derivation of Reported Contrasts

- E1→E2: 1.1957964897155762 − 1.1844764947891235 = +0.0113199949 (+0.9557%); time (65.22−33.26)/65.22 = 49.0% reduction.
- E2→E3: 1.2046293020248413 − 1.1957964897155762 = +0.0088328123 (+0.7387%).
- E3→E4: 1.2041741609573364 − 1.2046293020248413 = −0.0004551411 (−0.0378%).
- E2→E5: 1.1957060098648071 − 1.1957964897155762 = −0.0000904799 (−0.0076%).
- E3−A1: +0.0000057220 (A1 lower); E3−A2: +0.0000016689 (A2 lower). Both within observed repeat variation (~2×10^{-5}); not ranked.

D. References

(Citation keys match the source material in `paper/related_work.md`; verify bibliographic details before submission.)

[1] Wei Liu, Weihao Zeng, Keqing He, Yong Jiang, and Junxian He. "What Makes Good Data for Alignment? A Comprehensive Study of Automatic Data Selection in Instruction Tuning" (DEITA), ICLR 2024. arXiv:2312.15685.

[2] Zishang Jiang, Jinyi Han, Tingyun Li, Xinyi Wang, Sihang Jiang, Xiaojun Meng, Jiansheng Wei, Jiaqing Liang, and Yanghua Xiao. "Difficulty Is Not Enough: Curriculum Learning for LLMs Fine-tuning Must Consider Utility" (DUCL), AAAI 2026. DOI: 10.1609/aaai.v40i37.40400.

[3] Jing-Cheng Pang, Sun Liu, Chang Zhou, Xian Tang, Haichuan Ma, Kun Jiang, Jianlong Wang, Kai Zhang, Sijie Wu, Haoran Cai, Chenwei Wu, Xubin Li, and Xin Chen. "EDCO: Dynamic Curriculum Orchestration for Domain-specific Large Language Model Fine-tuning," ICML 2026. arXiv:2601.03725.

[4] Yushi Yang, Andrew M. Bean, Robert McCraith, and Adam Mahdi. "Evaluating Fine-Tuning Efficiency of Human-Inspired Learning Strategies in Medical Question Answering," NeurIPS 2024 Workshop on Fine-Tuning in Modern Machine Learning (FITML). arXiv:2408.07888.

[5] Bolin Zhang, Jiahao Wang, Qianlong Du, Jiajun Zhang, Zhiying Tu, and Dianhui Chu. “A Survey on Data Selection for LLM Instruction Tuning,” Journal of Artificial Intelligence Research 83:32, 2025. arXiv:2402.05123.

[6] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. “LoRA: Low-Rank Adaptation of Large Language Models,” ICLR 2022. arXiv:2106.09685.

[7] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. “Stanford Alpaca: An Instruction-following LLaMA Model,” 2023. GitHub repository: tatsu-lab/stanford_alpaca.